

Number: P3567R2
Date: 2024-12-16
Audience: EWG, LWG
Reply-to: [Hana.Dusiková](#)

Table of contents

- [Changes](#)
- [Timeline](#)
- [Quote](#)
- [Simple example](#)
- [Evaluation of constexpr coroutines](#)
- [Implementation experience](#)
 - [Alternative implementation approaches](#)
 - [Byte-code interpreter](#)
 - [C++ coroutines](#)
 - [AST transformation / split](#)
 - [Comparison](#)
- [Impact on existing code](#)
- [Intention for wording changes](#)
- [Proposed wording changes](#)
 - [Feature test macros](#)

constexpr coroutines

This paper is proposing making coroutines functional during constant evaluation. Even when most of use-cases for coroutines are based on I/O and event based, coroutines are still useful for compile-time based computation, eg. `std::generator`.

Changes

- [R1 → R2](#): Updated wording to elide allocator functionality in coroutine promise types. Added discussion about alternative implementation strategies. Added information about decision by various groups which seen this paper.
- [R0 → R1](#): Changed wording mentioning lifetime of coroutines after `expr.const` 5.2

Timeline

EWG in Wroclaw

Poll: P3367 constexpr coroutines send to CWG and LEWG for inclusion for C++26

SF	F	N	A	SA
4	12	3	0	0

Result: Consensus

LEWG in Wroclaw

Poll: Forward P3367R1 to LWG for C++26

SF	F	N	A	SA
12	5	0	0	0

Result: Unanimous consent

CWG in Wroclaw

Multiple implementers expressed concerns about resources needed for the